

Integration Testing

1 Introduction to Integration Testing

Integration testing is a systematic technique for constructing the software architecture while simultaneously testing interactions between integrated components.

Even if all modules pass unit testing, failures may occur when modules are combined due to interface and interaction issues.

1.1 Why Integration Testing Is Necessary

Problems commonly found during integration include:

- Data loss across interfaces
- Unexpected side effects between components
- Incorrect behavior when subfunctions are combined
- Magnified computational imprecision
- Issues with global data structures

The primary challenge in integration testing is **interfacing**, not individual logic.

2 Integration Strategies

2.1 Big Bang Integration

Big bang integration combines all components at once and tests the entire system as a whole.

Big bang integration is a lazy and high-risk strategy that often leads to chaos.

Problems with Big Bang Integration:

- Difficult error isolation
- Late discovery of interface defects
- Endless cycle of error correction

3 Incremental Integration

Incremental integration builds and tests the software in small, manageable increments.

Advantages:

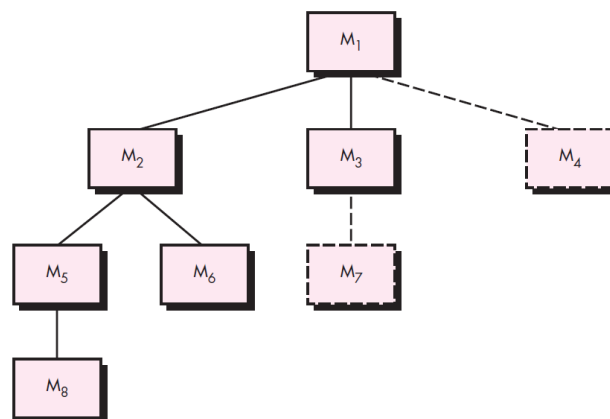
- Easier fault isolation
- More complete interface testing
- Controlled and systematic testing

Incremental integration strategies include:

- Top-down integration
- Bottom-up integration
- Sandwich (hybrid) integration

4 Top-Down Integration Testing

Top-down integration begins with the main control module and proceeds downward through the control hierarchy.



Modules are integrated either:

- Depth-first, or
- Breadth-first

4.1 Depth-First Integration

- Integrates all modules along a major control path
- Allows early demonstration of a complete function

4.2 Breadth-First Integration

- Integrates all modules at the same hierarchical level
- Expands horizontally across the architecture

4.3 Steps in Top-Down Integration

1. Use the main control module as a test driver
2. Replace subordinate stubs one at a time
3. Execute tests after each integration step
4. Perform regression testing

4.4 Advantages and Problems

Advantages:

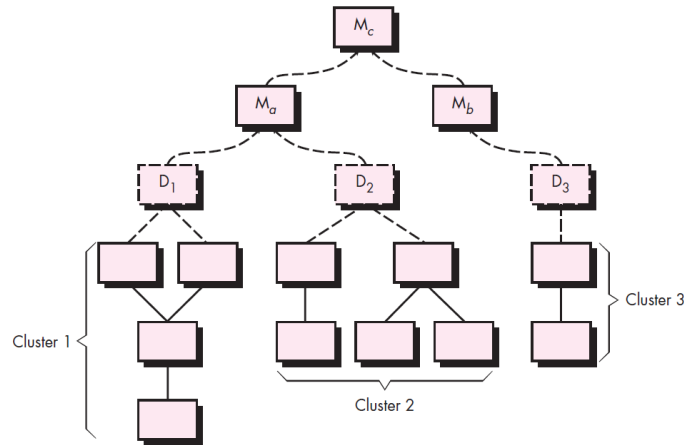
- Early testing of major control logic
- Early functional demonstrations

Problems:

- Heavy reliance on stubs
- Limited data flow at early stages
- Complex stub development

5 Bottom-Up Integration Testing

Bottom-up integration begins with low-level (atomic) modules and progresses upward.



5.1 Steps in Bottom-Up Integration

1. Combine low-level modules into clusters
2. Write drivers to test each cluster
3. Test clusters independently
4. Remove drivers and integrate upward

5.2 Advantages and Limitations

Advantages:

- No need for complex stubs
- Lower-level functionality always available

Limitations:

- System-level functionality appears late
- Difficult to demonstrate early progress

6 Regression Testing

Regression testing is the reexecution of previously conducted tests to ensure that changes have not introduced new defects.

Regression testing is essential because:

- New modules introduce new control paths
- Existing functionality may be affected

6.1 Regression Test Suite

A regression test suite includes:

- Tests covering all major software functions
- Tests affected by recent changes
- Tests for modified components

Regression testing reduces unintended side effects.

7 Smoke Testing

Smoke testing is a daily integration testing approach used to detect major defects early.

7.1 Smoke Testing Process

1. Integrate newly developed components into a build
2. Execute tests that expose showstopper errors
3. Rebuild and retest the entire system daily

7.2 Benefits of Smoke Testing

- Minimizes integration risk
- Improves overall product quality
- Simplifies error diagnosis
- Provides clear progress visibility

Smoke testing acts as a rolling integration strategy with daily feedback.

8 Strategic Integration Options

Top-down and bottom-up strategies each have trade-offs.

A hybrid approach (sandwich testing) often provides the best balance.

8.1 Critical Modules

A **critical module**:

- Addresses multiple requirements
- Has high control authority
- Is complex or error-prone
- Has strict performance constraints

Critical modules should be tested as early as possible.

9 Integration Test Work Products

Integration testing produces several key documents:

- Test Plan
- Test Procedure
- Test Specification
- Test Report

9.1 Integration Test Phases

Typical integration phases include:

- User interaction testing
- Sensor or input processing
- Communication functions
- Alarm or event processing

9.2 Common Evaluation Criteria

- Interface integrity
- Functional validity
- Information content
- Performance compliance

10 Summary

- Integration testing focuses on component interaction.
- Incremental integration is superior to big bang integration.
- Top-down and bottom-up strategies offer complementary benefits.
- Regression and smoke testing are essential for stability.
- Careful planning and documentation ensure effective integration.